Neuromodulation enhances dynamic sensory processing in spiking neural network models

AbdalQader AlKilany[1] and Dan F. M. Goodman[1]

[1] *Imperial College London*

Abstract

Neuromodulators allow circuits to dynamically change their biophysical properties in a context-sensitive way. In addition to their role in learning, neuromodulators have been suggested to play a role in sensory processing at relatively fast timescales (less than a second), although the precise mechanisms at play are still not well understood. To assess the potential computational role of neuromodulators in sensory processing, we added a simple but flexible model of neuromodulation to spiking neural networks. These networks were then trained – with methods from machine learning – to carry out challenging sensory processing tasks. We find that this addition leads to a dramatic improvement in sensory processing in every task and configuration we tested. In particular, we find that without explicitly training for this, it decreases reaction times, a role that has been discussed for the cholinergic system. In a particularly challenging speech recognition in noise task, we find that the networks learn to make use of rapid dynamic gain control via excitability, an attentional mechanism akin to the "listening in the dips" strategy. This has been hypothesised to be a key element of human hearing allowing us to perform better in these conditions than even state-of-the-art machine learning systems. We conclude that neuromodulation does have the potential to play a significant computational role in fast sensory processing. In addition, our neuromodulated spiking neural networks are able to substantially increase performance at only a small cost to computational complexity, and may therefore be valuable for applications in energy-efficient "neuromorphic" computing devices.

1 Introduction

In addition to relatively fast neurotransmitters acting at the level of synapses targeting a single cell, some neurons release neuromodulators that can change properties of small to large groups of cells [Nadim and Bucher, 2014, Mei et al., 2022]. These are usually studied at slow timescales, for example looking at their possible computational roles in learning [Doya, 2002, Grossman and Cohen, 2022]. Recent evidence, however, has suggested they may be important in sensory processing at faster time scales, even sub-second [Hangya et al., 2015, Bang et al., 2020]. A complete picture of the computational roles of neuromodulators, particularly at these fast timescales, remains elusive.

Here, we investigate how an abstract model of neuromodulation can improve rapid sensory processing in spiking neural networks. Simplified models of neuromodulation in spiking neural networks have been suggested before (e.g. Krichmar 2012), however recent algorithmic developments in training spiking neural networks allow us to study this at a larger scale and in the context of much more challenging real-world environments [Neftci et al., 2019, Zenke et al., 2021]. In previous work, we showed how a population of spiking neurons with heterogeneous time constants can have considerable computational advantages over a homogeneous network, and that the learned distributions of time constants are a good fit to experimental data [Perez-Nieves et al., 2021]. In this study, we take this idea further by making time constants (and other neural parameters) dynamically controllable by the network itself (via neuromodulation), allowing for more dynamic context-sensitive information processing. In brief, we find that neuromodulation enhances sensory processing across a range of challenging tasks, and has a particularly salient role in handling temporally complex signals (supporting and extending the conclusions of our previous study).

Our main motivation was to understand the potential computational role of neuromodulators in biological neural systems, however we will also discuss how neuromodulation could be valuable for the design of neuromorphic devices. Neuromorphic computing is the field devoted to using brain-inspired principles to inform the design of computing devices, often with the goal of extreme energy efficiency [Schuman et al., 2022, Kudithipudi et al., 2025]. Many of these devices are based on spiking neural networks, which have the potential for orders of magnitude improvements in energy efficiency compared to current machine learning methods, but are notoriously difficult to train to a high level of performance. While recent developments have improved the situation [Neftci et al., 2019], it is still challenging to train SNNs to the level of performance required for engineering applications.

2 Results

We started from the hypothesis that allowing networks of spiking neurons to dynamically adjust their own parameters in a context-sensitive way would improve their performance on tasks, particularly those with rich temporal structure (as in Perez-Nieves et al. 2021). To test this, we started from a simple baseline spiking neural network (SNN) consisting of a layer of spiking input neurons, connected to a recurrently connected hidden layer of spiking neurons, that was in turn connected to a linear readout layer. The hidden layer of spiking neurons include trainable parameters such as time constants, resting potentials and thresholds. In addition, all synaptic weights between layers were trainable. For training, we used the method of surrogate gradient descent [Neftci et al., 2019]. We used three datasets, including two spiking speech recognition datasets, and one visual gesture recognition dataset based on a neuromorphic event camera (DVS128 Gestures). Spiking Heidelberg digits (SHD) is a clean audio dataset with 700 audio channels, in which the task is to recognise spoken digits into one of twenty classes (digits 0-9 in English and German). Spiking speech commands is similar, but based on a noisier dataset with 35 words to recognise. For our baseline model, we use a relatively small network so that it is able to carry out the task at a decent level of performance but is not at the performance ceiling (fig. 2).

2.1 Neuromodulation enhances sensory processing

In our first model of neuromodulation, alongside the spiking neural network we add an artificial neural network that we refer to as the *neuromodulatory network*. It accumulates inputs from the hidden layer of the spiking networks for a fixed duration (K timesteps, where each timestep is 1 ms), and passes this input through a multilayer perceptron (MLP). The outputs of this MLP are used as the new parameter values (time constants, threshold, etc.) for the next simulation window of K timesteps. The whole process then repeats for the remainder of the input stimulus (fig. 1). We find that this leads to large improvements in performance across all datasets (fig. 2). For some datasets, updating the neuron parameters every time step led to the largest performance improvement, for others it was optimal to update them every 50 to 200 ms. The trade-off is between having more time to integrate more contextual information, versus allowing for a more temporally fine grained modulation.

Note that we did not attempt to directly control the number of parameters in the comparisons we make, but since all our networks consist of a number of neurons that gives saturated performance, increasing the number of neurons without adding neuromodulation would not have improved performance (fig. 2a). All performance improvements therefore are due to the change in model and architecture, and not solely to the increase in the number of parameters.

We also tested what happens when instead of replacing the neuron parameters, the neuromodulatory network can only modify them (increase or decrease them). This may be more plausible as a biophysical model. We find that in this case, performance is still improved in all conditions, but not as much as when parameters are replaced (fig. 2).

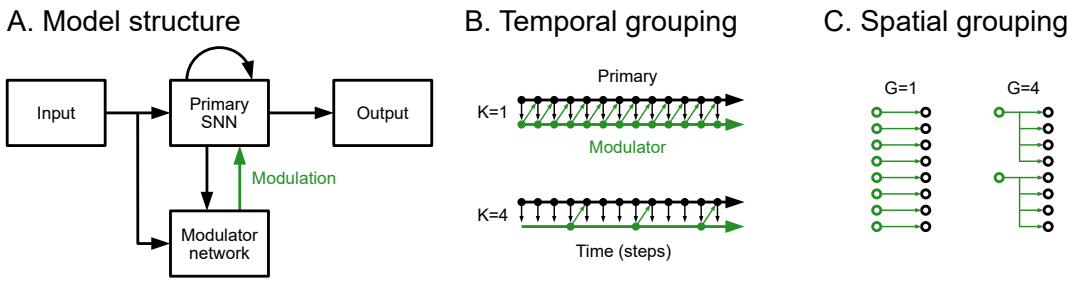

Figure 1: **(A)** Model structure. The primary spiking neural network (SNN) processes input spike trains, while a secondary modulator network adjusts parameters of the primary network based on the context. The output of the primary SNN is used to classify the input. **(B)** Temporal grouping. When $K = 1$ the modulator network receives input at every time step and modifies parameters of the primary SNN at every time step. When $K > 1$ the modulator network runs every K time step, receiving all the inputs from the previous K time steps and modifying the parameters of the primary SNN for the subsequent K time steps. **(C)** Spatial grouping. If the group size $G = 1$ then every output of the modulator modifies a single parameter of a single neuron. When $G > 1$ each output modifies a single parameter of G neurons identically.

The effect of neuromodulators can be as specific as a single neuron, or may impact a small or large group of neurons. We therefore tested the effect on performance when neurons are grouped into smaller or larger groups, with each group having the same neuromodulatory effect. At least in the tasks and networks we tried, spatially extended neuromodulation appeared to be about equally effective as highly specific neuromodulation (fig. 2), although this may be different for different tasks.

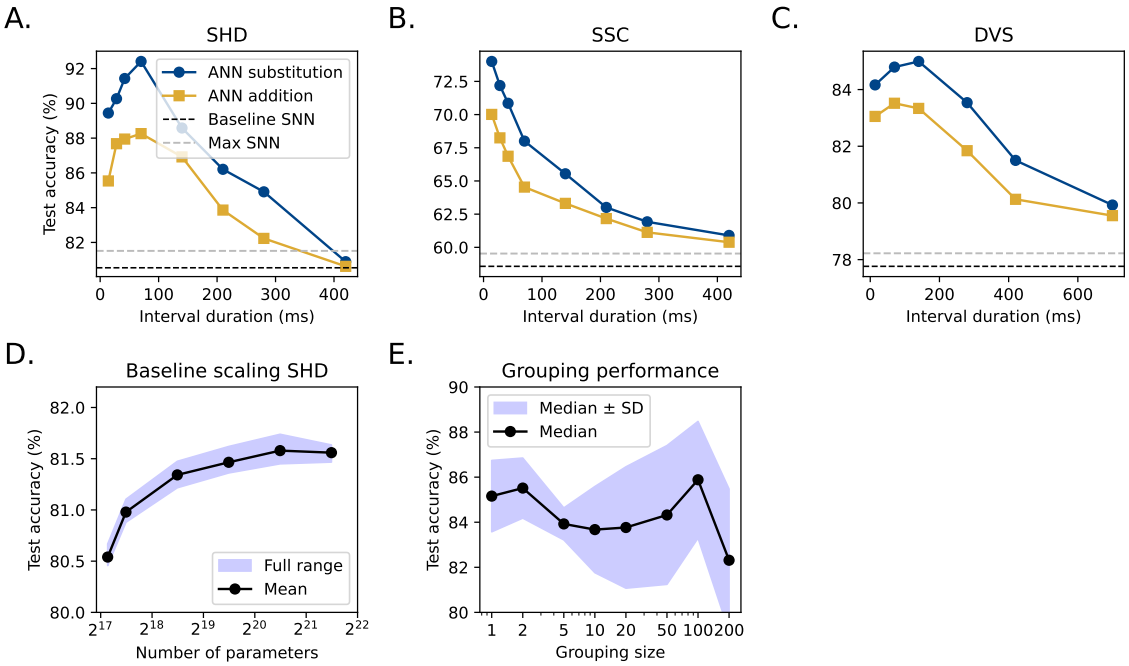

Figure 2: **(A-C)** Performance of modulated SNN on Spiking Heidelberg Digits (SHD), Spiking Speech Commands (SSC) and Dynamic Vision Sensor (DVS) datasets when the modulation network runs at different time intervals. Performance is shown for an artificial neural network modulator using the substitution and addition methods. **(D)** Test accuracy of unmodulated SNNs as a function of total parameter count (log scale). **(E)** Performance as a function of spatial grouping size (ANN addition method on SHD dataset).

So far, we have used an artificial neural network to control the parameters of a primary spiking neural network. We next tested the effect of replacing this artificial network with a spiking network. In this case, the entire model can be considered as a single spiking neural network where some neurons have an effect on the membrane potential of their outputs, and some have an effect on parameters. Note that in this case we cannot easily use the substitution method, but something closer to the additive method, where each output spike causes an increase or decrease in the value of some parameter. We find that the spiking network-based neuromodulation performs at least as well in all cases as the ANN addition version (fig. 3). In some cases (DVS dataset, and for longer intervals in the SSC dataset) the SNN-based version performs better than the ANN substitution network, and in others vice versa (SHD dataset, shorter intervals in the SSC dataset).

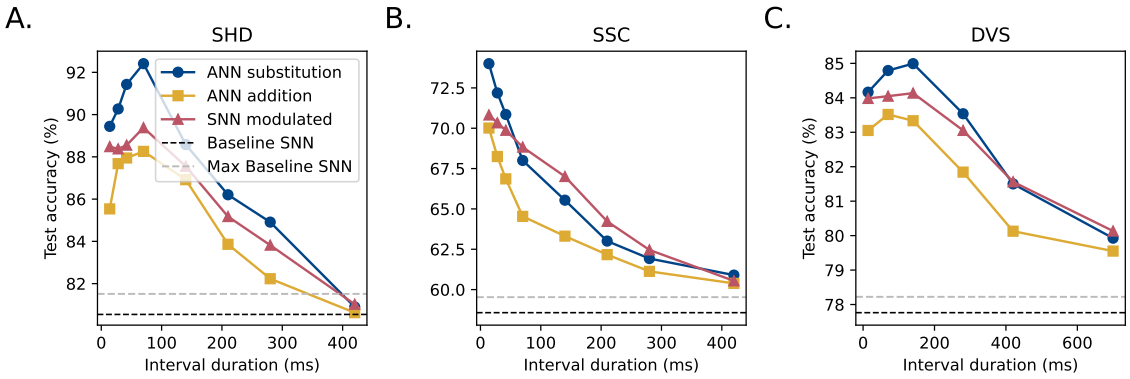

Figure 3: **(A)** Comparison of artificial (ANN) and spiking neural network (SNN) controller on Spiking Heidelberg Digits (SHD) dataset. **(B-C)** Same plot but for the Spiking Speech Commands (SSC) and Dynamic Vision Sensor Gestures (DVS128) datasets.

2.2 Neuromodulation improves reaction times

It has been suggested that neuromodulators may play a key role in controlling decision-making and reaction times [Grossman and Cohen, 2022]. In our model, we measured reaction times indirectly, by defining the decision time of the network as the time at which output activity is at a peak (fig. 4). This is a simplification, because it is acausal – the network cannot know in advance that it has reached its peak activation – however, it would follow the same trend as a model based on decision thresholds, for example.

We find that in our simulations, without directly training for reaction times, decisions are made around 10% earlier (mean, or 17% earlier median) in the modulated versus unmodulated networks (fig. 4). The distribution of reaction times in the unmodulated networks is roughly unimodal (around 730 ms), while in the modulated networks appears to have two peaks (primary around 400ms, secondary around 900ms). This suggests that neuromodulation allows the network to respond dynamically to the most informative parts of the signal, and when those are early it can make its decision immediately, but with the flexibility to decide later if necessary.

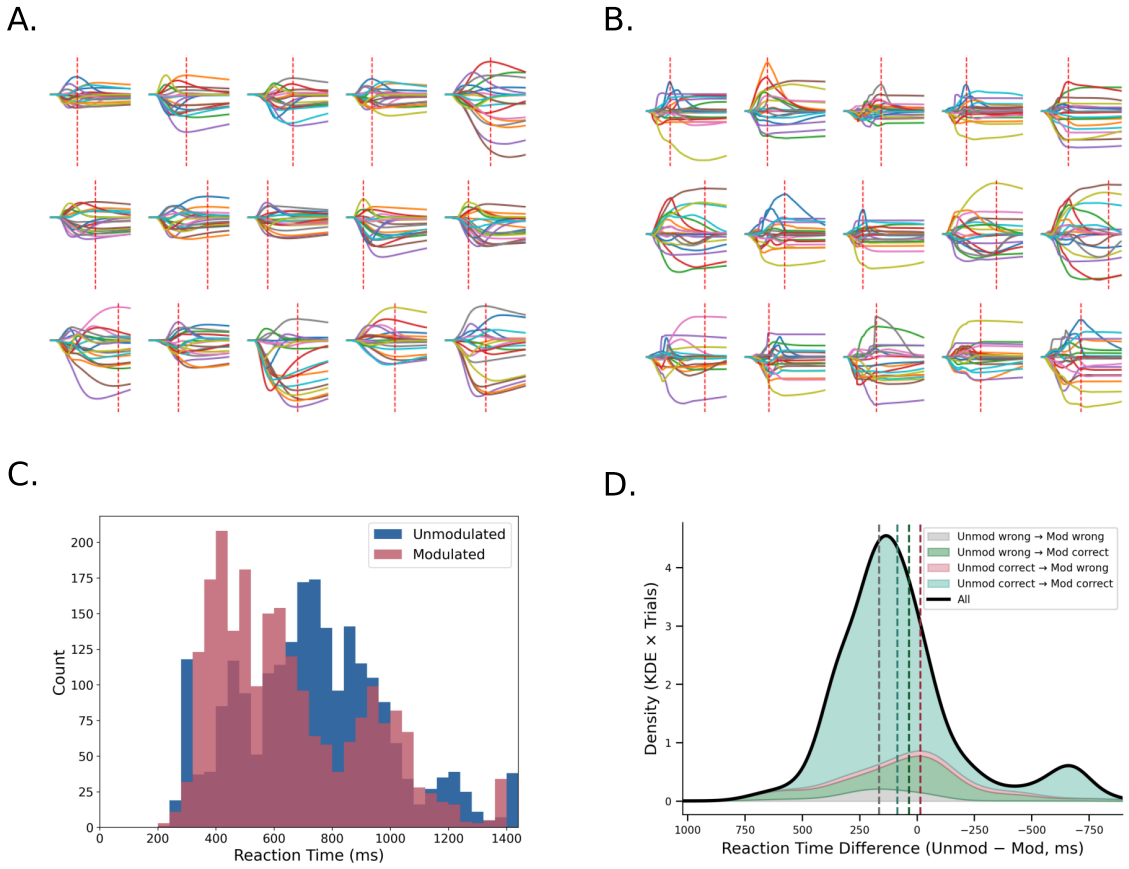

Figure 4: **(A, B)** Voltage traces (x=time, y=membrane potential) from a random selection of hidden units in the unmodulated (A) and modulated (B) primary spiking neural network (SNN). Vertical red dashed lines indicate the peak where the 'decision' is made. **(C)** Decision time distribution for unmodulated and modulated SNNs. The unmodulated SNN has a mean of 734 ms and median of 728 ms. The modulated SNN is faster on average (mean: 656 ms, median: 602 ms), but also exhibits a secondary peak later in the trial. **(D)** Change in decision time (unmodulated minus modulated) grouped by outcome (grey: wrong→wrong; green: wrong→correct; red: correct→wrong; teal: correct→correct). Most data points show faster reaction times after modulation, but there is a second bump between –500 and –750 ms where modulation slowed some responses, and a relatively small group of corrected trials (green) were slower.

2.3 Temporally dynamic gain control improves effective signal to noise ratio

The bimodal distribution of decision times in the modulated network suggests that it is able to dynamically increase (or reduce) the sensitivity of the response at times where the signal is most (least) informative. To test this in a more challenging scenario, we created a new dataset based on the SHD dataset, but with a time-varying background noise. We tested this in either a sinusoidally amplitude-modulated (SAM) background noise at different amplitude modulation frequencies, or a natural noise background. We find that the modulated network performs substantially better (fig. 5), with accuracy up to 25% higher, particularly at higher levels of background noise (below -10 dB SNR). For example, at around -17 dB SNR accuracy increased from about 35% to almost 60%.

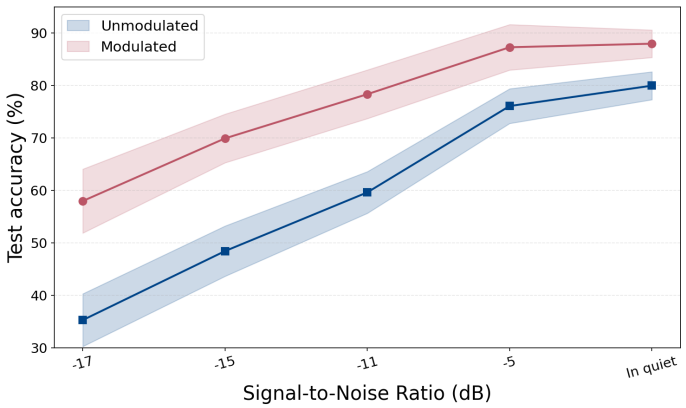

Figure 5: Test accuracy of the unmodulated and modulated networks across varying signal-to-noise ratios (SNRs). Modulation improves performance in low-SNR conditions. The performance gap between the two models widens as noise increases from around 10% in quiet to approximately 25% at −17 dB SNR.

It has been speculated that the human auditory system is able to "listen in the dips", that is, focus attention on those possibly brief moments where the noise is relatively low compared to the speech signal. We verified that our model seems to be using this strategy by comparing the network's firing rates at moments of low versus high noise level in the modulated and unmodulated networks (fig. 6). By analysing how neural parameters change during the signal, we find that our model significantly modulates all parameters, with the clearest effect at high noise levels being to increase the firing thresholds and decrease reset values (fig. 7).

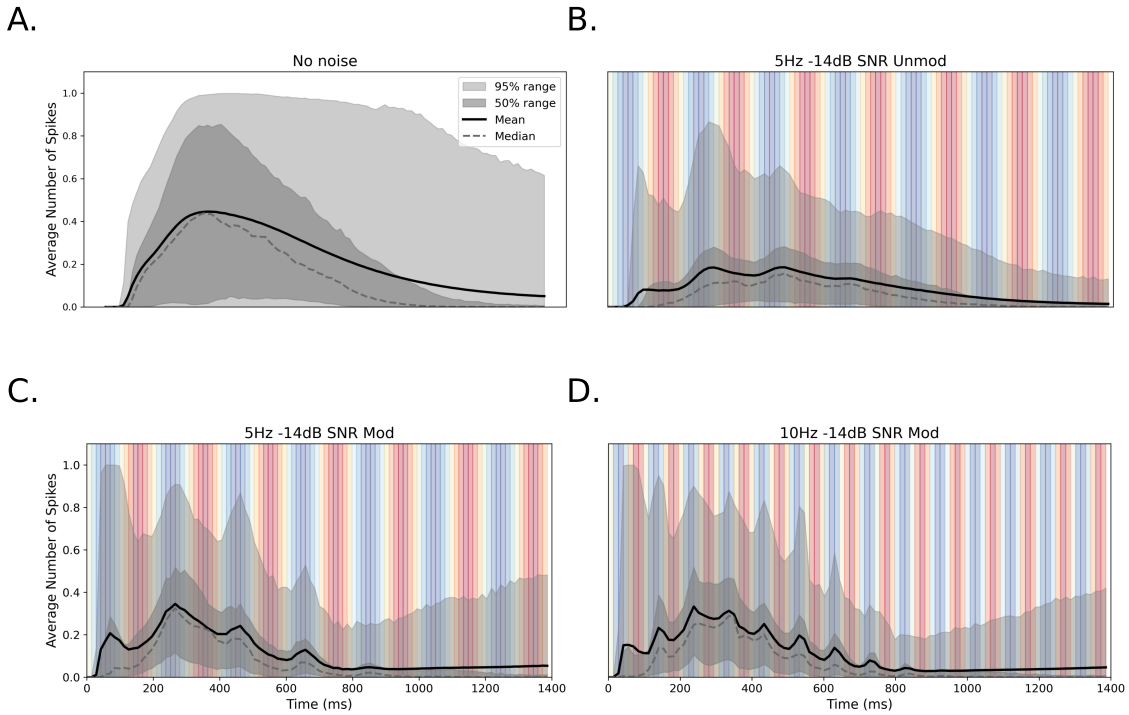

Figure 6: Average spike activity across four network conditions. **(A)** Unmodulated network in silence (no background noise); firing rates vary steadily. **(B)** Unmodulated network with sinusoidally amplitude modulated (SAM) background noise (modulation frequency 5Hz, -14dB SNR); firing rate adapts slightly to noise level (blue=low noise amplitude, red=high noise amplitude). **(C)** Modulated network in the same conditions as (B); spiking is noticeably suppressed during noisy peaks (red) and elevated during quiet dips (blue). **(D)** Same modulated network as (C), trained at 5 Hz modulation frequency but tested at a mismatched 10 Hz modulation frequency. The firing rate modulation persists despite the change in modulation frequency.

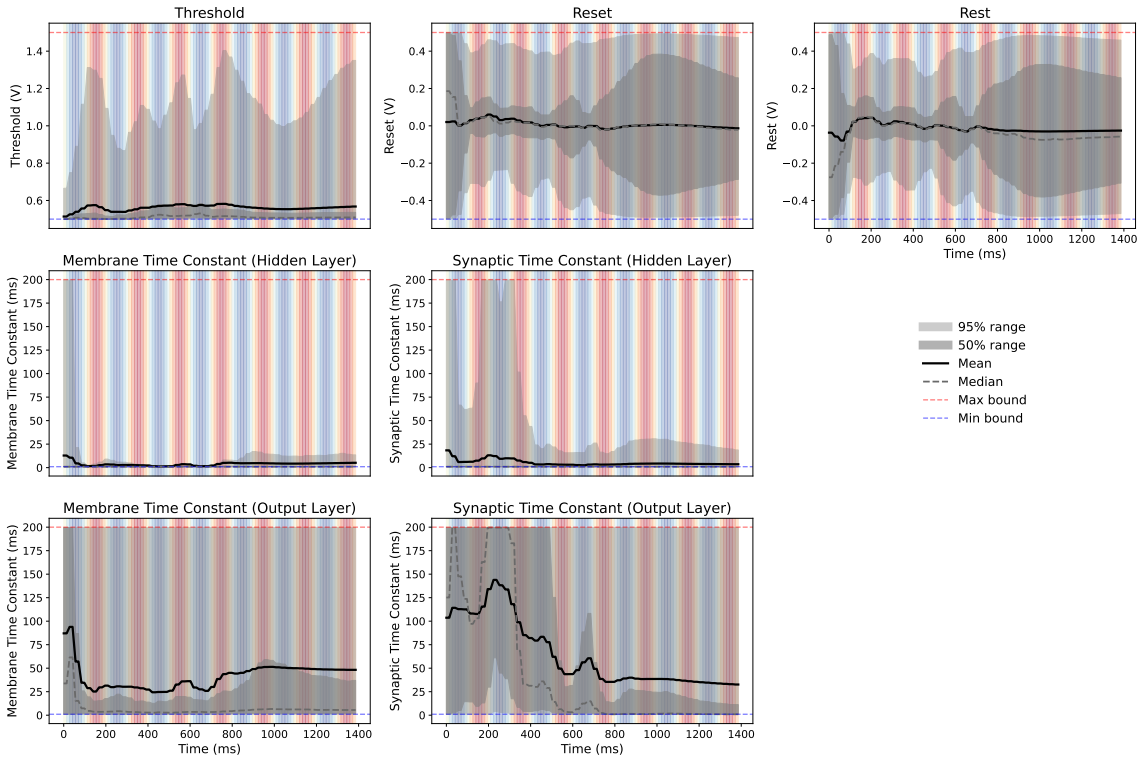

Figure 7: Adaptation of neuron parameters in the modulated network during sine wave noise. Each panel shows the evolution of a learned parameter over time, with thick black lines denoting the mean across trials and shaded regions indicating 50% and 95% quantiles. The top row displays spiking threshold, reset, and resting potential. The middle and bottom rows show membrane and synaptic time constants for the hidden and output layers, respectively. A sinusoidal background is overlaid in each panel, where red indicates peaks and blue indicates troughs in the sinusoidal noise input. The red and blue dashed lines mark the maximum and minimum parameter bounds.

To test that the network was using an adaptive strategy, and not just learning the characteristics of a specific noise we trained on, we varied starting phase of the SAM noise, as well as training at one frequency and testing at another frequency (fig. 6). We also tested on a natural and unpredictable noise sample and found that network spike rate negatively correlates with noise amplitude in this sample (fig. 8).

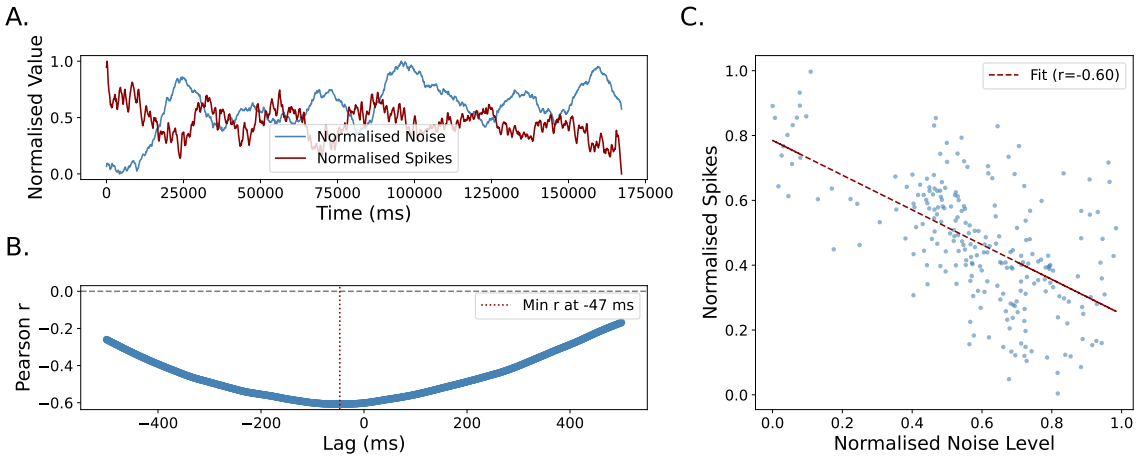

Figure 8: **(A)** Smoothed and normalised background noise level (blue) and average spike rate (red) over time. The network increases activity during quieter periods, showing an inverse relation between spike output and natural noise level. **(B)** Cross-correlation between spike rate and noise level shows that the strongest modulation effect lags by roughly 50 ms. **(C)** Normalised spike rate against noise level for 1 second windows, showing a strong negative correlation ($r = -0.60$).

3 Methods

The general structure of the model is shown in fig. 9A. A neural network is trained to classify a set of inputs represented as spike trains. The network takes as inputs a sequence of spike trains, which are fed into a primary spiking neural network (SNN). The output of this network is fed into a linear output layer that is used to classify the inputs with some loss function. The errors from this loss function are backpropagated to update the parameters of the network. In addition, a secondary network is trained to modulate the parameters of the primary SNN based on the input. This secondary network can be either an artificial neural network (ANN) or another SNN. It outputs either full parameter values or additive adjustments to existing parameters. It can be set to update parameters every time step of the simulation, or with a fixed interval. The network can be set up so that every parameter can be modulated independently, or parameters can be grouped together to be modulated as a single unit. The parameters that can be modulated include the spiking threshold, resting potential, membrane time constants, and synaptic time constants.

3.1 Models

3.1.1 Spiking neurons

The spiking neurons are modeled as leaky integrate-and-fire (LIF) units. Each unit has a membrane potential that evolves over time according to the following differential equations:

$$\begin{aligned} \tau \dot{v} &= -v + x \\ \tau_x \dot{x} &= -x \end{aligned} \tag{1}$$

Here, v is the membrane potential, x is a synaptic variable, τ is the membrane time constant, and τ_x is the synaptic time constant. When a spike arrives at the neuron from a synapse with a weight w, the following event update is applied:

$$v \leftarrow v + w \tag{2}$$

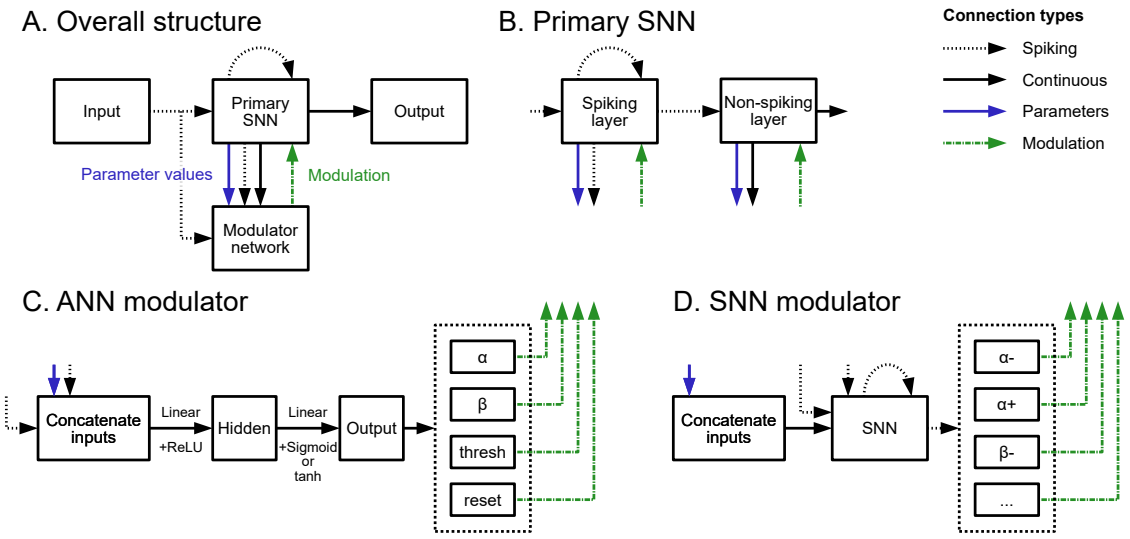

Figure 9: **(A)** Global architecture. The primary spiking neural network (SNN) processes input spike trains, while a secondary modulator network adjusts parameters of the primary network based on the context. The output of the primary SNN is used to classify the input. **(B)** Primary SNN architecture. Input spikes arrive at a recurrent layer of spiking neurons. Spikes from this layer are sent to a layer of non-spiking but leaky neurons. Outputs from both layers are sent to the modulator, and from the non-spiking layer as used as output for the classifier. **(C)** ANN-based modulator architecture. Inputs are concatenated into a single vector, passed through a linear + ReLU to a hidden layer, which is passed through a linear + sigmoid output layer (or linear + tanh). This layer is divided into different parameters that are used to modulate the primary network. **(D)** SNN-based modulator architecture. Continuous valued inputs are concatenated and passed as currents to the SNN, while input spikes are directly fed to a recurrent spiking layer. The spiking outputs of this layer are divided into groups that either increase or decrease a corresponding parameter of the primary network.

When the membrane potential exceeds a threshold, the neuron emits a spike and resets its potential:

$$\begin{aligned} \text{if threshold exceeded} \quad & v > v_{\text{th}} \\ \text{emit spike} \quad & \\ \text{and reset} \quad & v \leftarrow v_r \end{aligned} \tag{3}$$

Note that for simplicity of the implementation, in the code the time constants τ and τ_x are not directly used, but instead the parameters α and β are used, which are defined as:

$$\begin{aligned} \alpha &= e^{-\mathrm{d}t/\tau_x} \\ \beta &= e^{-\mathrm{d}t/\tau} \end{aligned} \tag{4}$$

3.1.2 Artificial neurons

The artificial neurons are modeled as multi-layer perceptrons (MLPs). Each layer of neurons takes as input a vector of activations $\mathbf{x}$ and computes an output vector $\mathbf{y}$ using the following equation:

$$y = f(Wx + b) \tag{5}$$

Here, W is the weight matrix, b is a bias vector, and f is an activation function. We use three different activation functions. The rectified linear unit (ReLU):

$$f(x) = \mathrm{ReLU}(x) = \max(0, x) \tag{6}$$

The sigmoid function:

$$f(x) = \sigma(x) = \frac{1}{1 + e^{-x}} \tag{7}$$

And the hyperbolic tangent function:

$$f(x) = \tanh(x) \tag{8}$$

3.1.3 Neuromodulation

Our simple model of neuromodulation allows the secondary modulator network to change the value of any parameter of the primary network. There are two ways to do this. The first (which we refer to as *substitution*) is to output the full value of the parameter, which is then used directly in the primary network. So if the parameter is p and the output of the modulator network is m then

$$p \leftarrow m \tag{9}$$

The second method (*addition*) is to output an additive adjustment to the existing parameter value, which is then added to the current value of the parameter in the primary network.

$$p \leftarrow p + m \tag{10}$$

The first method can only be used with an artificial neural network modulator, while the second method can be used for either artificial or spiking modulator networks. In the case of a spiking modulator network, we use two output neurons for each parameter: one for the positive adjustment and one for the negative adjustment. The size of the adjustment for each spike is a non-negative learnable parameter.

In the case of the substitution method, the modulator network is designed to output only valid values for the parameters. In the case of the addition method, this is not possible, and we therefore apply clipping after each update to ensure that values stay within valid ranges.

3.2 Architectures

3.2.1 Primary SNN

The primary SNN (fig. 9B) consists of a single recurrent hidden layer of leaky integrate-and-fire neurons, connected to a non-recurrent readout layer of leaky but non-spiking neurons (identical equations but no threshold and reset mechanism). Every connection (input to hidden, hidden to hidden and hidden to output) is fully connected and represented by a corresponding weight matrix.

3.2.2 Modulator ANN

The ANN modulator network (fig. 9C) consists of a two layer multi-layer perceptron (MLP). The first layer is a linear layer with a ReLU activation function, and the second layer is linear with a sigmoidal activation function (in the case of the substitution method, to keep parameter values within the desired range) or tanh activation function (in the case of the addition method, to allow values to be positive or negative but bounded). For the substitution method an offset is applied to the output for some parameters (+0.5 for threshold, -0.5 for rest and reset).

The network receives as input a concatenated vector of the current values of all the parameters it can modulate, as well as the recent spiking activity of the primary SNN and the input spike trains. In the case where the modulator network is set to update every K time steps, the input spike activity is summed over the last K time steps (see section 3.3.1 for details).

3.2.3 Modulator SNN

The SNN modulator network (fig. 9D) consists of a single recurrent layer of leaky integrate-and-fire neurons. The network receives the same inputs as for the ANN modulator, but in this case they are treated as input currents to the neurons. Each output neuron corresponds to a positive or negative change to a specific parameter value. All weight matrices and the size of the output changes are learnable.

3.2.4 Architecture parameters

Parameter	SHD	SSC	DVS	Description
Input neurons	700	700	4096	Number of input units
Hidden spiking neurons	200	200	200	Size of hidden LIF layer
Output neurons	20	35	11	Number of classes
Learning rate	2×10^{-4}	2×10^{-5}	2×10^{-4}	Initial optimizer learning rate
τ_m		20 ms		Membrane time constant
τ_s		10 ms		Synaptic time constant
U_{th}		1 V		Spiking threshold
U_0		0 mV		Resting potential
U_r		0 mV		Reset potential
t_{ref}		0 ms		Refractory period
ρ		100		Surrogate-gradient steepness
Time constant min		1 ms		Minimum τ_m, τ_s
Time constant max		200 ms		Maximum τ_m, τ_s
Threshold min		0.5 V		Minimum U_{th}
Threshold max		1.5 V		Maximum U_{th}
Reset & rest min		-0.5 V		Minimum U_0, U_r
Reset & rest max		0.5 V		Maximum U_0, U_r
Optimizer		Adam		Optimization algorithm
Batch size		64		Training batch size

Table 1: Model architecture parameters, grouping dataset-specific hyperparameters (top) and core biophysical parameters with clipping ranges plus common settings (bottom).

3.3 Grouping

3.3.1 Temporal grouping

We tested the effect of running the ANN modulator network after each K time steps of the simulation, instead of every time step. To ensure that no information was lost, the input spike activity was summed over the last K time steps before being fed into the modulator network. The output of the modulator network was then applied to the primary SNN parameters every K time steps.

The SNN modulator network was always run at every time step, but the effect on the modulated parameters is only updated every K time steps.

3.3.2 Spatial grouping

In spatial grouping we use the same modulator output for a group of G neurons. For comparability, we only use this for the ANN addition-based modulator. Parameter values at the start of the simulation are a learnable parameter, and heterogeneous (different for each neuron). However, an

increase or decrease in a parameter value from the modulator network during a trial will be shared between a group of G neurons. This ensures that the network remains fully heterogeneous (which would not be possible with the ANN replacement method).

3.4 Training

All networks were trained using surrogate gradient descent [Neftci et al., 2019], which, in the backwards pass only, replaces the non-differentiable spike function with a smoothed sigmoid to enable backpropagation. Specifically, in the forward pass we use the Heaviside function

$$H(x) = \begin{cases} 1 & \text{if } x > 0 \\ 0 & \text{if } x \leq 0 \end{cases} \tag{11}$$

in the threshold equation $S = H(v - v_{\text{thr}})$, where $S = 1$ indicates a spike. When computing gradients in the backwards pass, we replace $H'(x)$ with $1/(|x| + 1)^2$ as in Zenke and Ganguli [2018].

We used the Adam optimiser [Kingma and Ba, 2017] with a learning rate of 10^{-3}. The following parameters are learnable. For the primary SNN: the input-to-hidden weight matrix, the hidden-to-output weight matrix, the hidden-to-hidden recurrent weight matrix, and the initial value of each parameter for each neuron (membrane and synaptic time constants, threshold, reset, rest). For the ANN modulator: the weights and biases for each layer. For the SNN modulator: the input-to-hidden weight matrix, the sizes of the effects that each output spike has on the corresponding parameter (one parameter per parameter per neuron), and the SNN parameters (same as for the primary SNN).

Training was split into two phases:

1. Pre-training the primary SNN without modulation. We fully train this network and use it both as the unmodulated baseline for comparison, and to provide the initial values for the modulated network in the second phase. We found this was necessary to obtain good performance in the modulated network. In more detail, for SHD and DVS we determine a training loss threshold based on an initial training run that is subsequently discarded. This training loss threshold is set as the training loss at the point where the test loss starts to increase. In subsequent runs we do not use the test loss as part of the stopping condition, only the training loss threshold. For the SSC dataset we use the validation set to determine when to stop training.
2. Joint training with the modulator network. We train in the same way as for the pre-training phase (but with a correspondingly lower training loss threshold), but using the pre-trained primary network as initial parameter values.

The loss function consisted of three terms. A task loss and two regularisation terms.

Let $v_i(t)$ be the membrane potential of output neuron i at time t. We define the output vector

$$v_i^{\max} = \max_t v_i(t).$$

The output logits are then set to

$$x_i = \text{softmax}(\mathbf{v}^{\max}) = \frac{\exp(v_i^{\max})}{\sum_j \exp(v_j^{\max})}.$$

The task loss is then the cross-entropy loss on these logit values.

The regularisation terms keep spike rates within reasonable ranges and help to avoid instability in training. There are two terms. The first term is proportional to the sum over the neuron and batch indices of $(r - 0.01)^2$ where r is the neuron's firing rate. The second term is proportional to the sum over the batch of $\text{relu}(r - 100)^2$ where r is the population spike count.

Parameter values were clipped to stay within biologically plausible ranges: time constants were clipped between 1-200 ms; thresholds were clipped between 0.5 and 1.5.

3.5 Datasets

3.5.1 Standard datasets

We evaluated our models on three datasets:

- **Spiking Heidelberg Digits (SHD):** A clean audio digit dataset of ten spoken digits by 12 distinct speakers in English and German, processed via a model of the cochlea into 700-channel spike trains [Cramer et al., 2022]. The dataset consists of 20 classes, 8156 training samples, and 2264 testing samples. Two of the speakers are only present in the test set.
- **Spiking Speech Commands (SSC):** A more complex and noisy speech dataset with 35 classes, many different speakers, and diverse recording conditions from the Google Speech Commands dataset [Warden, 2018] processed into spike trains in the same way as SHD. The dataset consists of 35 classes, 75466 training samples, 9981 validation samples, and 20382 testing samples.
- **DVS128 Gestures:** An event-based video gesture dataset captured using a $128 \times 128 = 16384$ pixel dynamic vision sensor [Amir et al., 2017]. Contains 11 gestures from 29 subjects under 3 illumination conditions. 23 subjects are used for the training set, and 6 for the testing set. In total, there are 1342 samples in the dataset.

3.5.2 Speech-in-noise dataset

For the speech in noise experiments we used the following process. We took the original target audio files used in the SHD dataset, added either sinusoidally amplitude-modulated (SAM) noise or natural noise, and then converted into spike trains using the same technique as in Cramer et al. [2022].

Note that after correspondence with the authors, we found that the code available online at https://github.com/electronicvisions/lauscher does not exactly correspond to what was used in the paper. To reproduce their results we had to modify their code to reduce the size of the Hanning window to a 30 ms ramp-up and ramp-down time.

The SAM noise is simply computed by multiplying a white noise (Gaussian distributed samples) with an envelope

$$(A/2)(1 + \sin(2\pi f_m t + \phi))$$

where A is the amplitude, f_m is the modulation frequency and ϕ is the starting phase. For the plots in the paper, we set $\phi = 0$ to make it possible to visualise the effect, but in the training and testing we randomise ϕ uniformly in $[0, 2\pi)$.

For the natural noise, we use a freely available sample of coffee shop noise[1]. We discard the first 50 ms and last 100 ms which have unusual characteristics, and use a segment of the noise with duration equal to the target sound duration.

In the case of both types of noise, we use a range of signal-to-noise ratios (SNRs). We use the target sounds calibrated in the same way as in Cramer et al. [2022], only modifying the amplitude of the noise to vary the SNR.

3.6 Reaction times

We did not use a detailed model of decision-making, and we measure "reaction times" by an indirect measure. As in section 3.4, we set $v_i(t)$ to be the output membrane potential of neuron i at time t, and define the reaction time to be

$$\mathrm{RT} = \mathrm{argmax}_{i,t}\, v_i(t).$$

[1] https://www.youtube.com/watch?v=Yat3wNuUG7A&t=35s

Since the maximum value of the membrane potential determines which output class is predicted, on a given trial anything that happens for $t > \text{RT}$ will not have changed the decision that would have been taken if the network had been forced to make a decision at time $t = \text{RT}$. In other words, our measure of the reaction time is the earliest time at which the decision became stable.

3.7 Reproducibility and code

All code and experiments were implemented in PyTorch [Paszke et al., 2019]. The code and reproducibility instructions are available at `https://github.com/abdalalkilani/NeuromodulationSNN`.

4 Discussion

We added a simple and flexible model of neuromodulation to a trainable spiking neural network model of perceptual processing (auditory and visual). This gave rise to a substantial improvement in performance across a wide range of tasks and conditions. Our results suggest that this approach may be widely applicable both in neuromorphic computing and neuroscience. We elaborate on these below.

For neuromorphic computing, our model has some key properties that will be useful in pursuing the goal of scaling these models up to the point where they can be used in real-world engineering problems. Firstly, the method is simple to implement, effectively just allowing neurons to modify any parameter of a neuron instead of just the membrane potential. Secondly, it leads to a large gain in performance at a small cost in terms of the number of parameters (and therefore memory). Thirdly, we have shown that it can be used flexibly, either via an artificial or spiking neural network, and at a range of spatial and temporal scales. Together, these properties allow us to customise the model for optimal efficiency on each specific design of neuromorphic device.

In terms of neuroscience, our aim was to study the possible computational role for rapid changes in neuromodulation in sensory processing [Hangya et al., 2015, Bang et al., 2020]. Our motivation for using spiking neural networks carrying out challenging tasks is that to understand any mechanism with a complex computational role we need to use models that meaningfully represent both the mechanism and the associated computational role. If either the model or the task is too simple, we cannot effectively investigate the connection between them. Our approach in this paper aims to fulfil this requirement with the least complicated combination of model and task that can be feasibly trained (spiking neural network and richly temporally structured sensory data). This allows us to probe the computational role of neuromodulation without overcomplicating the model and task design to the point where it is impractical to analyse the results.

Our results show that neuromodulation allows for a substantial improvement in performance at a relatively low energy cost, in that adding a modulatory network gives a significantly higher effect on performance compared to adding a similar number of parameters to an unmodulated network. It works well across all tasks and conditions we tested, suggesting that computationally it is a robust general purpose mechanism, and lending weight to the hypothesis that the neuromodulatory system may play this computational role (along with many others). This raises the question of what specific types of computation could neuromodulation be good for?

We found that neuromodulation of SNNs seems to allow the network to make decisions at earlier times if those are the most informative moments in signals. By combining this with an additional signal representing the balance of required accuracy versus speed, our model of neuromodulation could be important in flexible decision-making, consistent with suggestions that a number of neuromodulators may be implicated in this [Grossman and Cohen, 2022].

In a more challenging speech in noise task, we found that without biasing the training, it discovered an adaptive gain control strategy of 'listening in the dips' that has been suggested to be a key part of the human ability to perceive speech in noisy environments [Peters et al., 1998, Lorenzi et al.,

2006]. This is an ability that even state-of-the-art machine listening systems still struggle with [O'Shaughnessy, 2024]. In terms of the comparison with machine learning, one way of looking at the adaptive gain control solution found by our model is that it is a form of rapid attentional mechanism that focuses processing on the most informative parts of a signal. Attention is a key mechanism in the Transformer architecture that heralded the large language model revolution in machine learning [Vaswani et al., 2017], as well as being another hypothesised role of neuromodulation [Avery and Krichmar, 2017, Grossman and Cohen, 2022].

Our model has shown that neuromodulation can have an advantageous role in flexible decision-making and sensory processing in noise via an attentional mechanism. However, this does not exhaust the possible computational roles of neuromodulation in sensory processing. Firstly, there may be other sensory processing tasks in which neuromodulation would be advantageous. Secondly, our model uses a simplified general neuromodulatory mechanism in which the output of some neurons can modify various neuronal parameters such as excitability and time constants. This is consistent with known properties of neuromodulators, but not exhaustive [Nadim and Bucher, 2014]. In particular, we do not consider different types of neuromodulators, nor target-cell-type specific effects, nor interactions between different neuromodulators, all of which are known to be important in the nervous system [Grossman and Cohen, 2022]. In future work, it would be interesting to add additional biophysical constraints and features to our model, and to test these in an even wider range of sensory processing tasks. This would enable us to further probe the potential computational roles of specific neuromodulatory mechanisms.

References

Farzan Nadim and Dirk Bucher. Neuromodulation of Neurons and Synapses. *Current opinion in neurobiology*, 0:48–56, December 2014. ISSN 0959-4388. doi:10.1016/j.conb.2014.05.003. URL https://www.ncbi.nlm.nih.gov/pmc/articles/PMC4252488/.

Jie Mei, Eilif Muller, and Srikanth Ramaswamy. Informing deep neural networks by multiscale principles of neuromodulatory systems. *Trends in Neurosciences*, 45(3):237–250, March 2022. ISSN 0166-2236. doi:10.1016/j.tins.2021.12.008. URL https://www.sciencedirect.com/science/article/pii/S0166223621002563.

Kenji Doya. Metalearning and neuromodulation. *Neural Networks*, 15(4):495–506, June 2002. ISSN 0893-6080. doi:10.1016/S0893-6080(02)00044-8. URL https://www.sciencedirect.com/science/article/pii/S0893608002000448.

Cooper D. Grossman and Jeremiah Y. Cohen. Neuromodulation and Neurophysiology on the Timescale of Learning and Decision-Making. *Annual Review of Neuroscience*, 45:317–337, July 2022. ISSN 1545-4126. doi:10.1146/annurev-neuro-092021-125059.

Balázs Hangya, Sachin P. Ranade, Maja Lorenc, and Adam Kepecs. Central Cholinergic Neurons Are Rapidly Recruited by Reinforcement Feedback. *Cell*, 162(5):1155–1168, August 2015. ISSN 0092-8674, 1097-4172. doi:10.1016/j.cell.2015.07.057. URL https://www.cell.com/cell/abstract/S0092-8674(15)00973-3. Publisher: Elsevier.

Dan Bang, Kenneth T. Kishida, Terry Lohrenz, Jason P. White, Adrian W. Laxton, Stephen B. Tatter, Stephen M. Fleming, and P. Read Montague. Sub-second Dopamine and Serotonin Signaling in Human Striatum during Perceptual Decision-Making. *Neuron*, 108(5):999–1010.e6, December 2020. ISSN 0896-6273. doi:10.1016/j.neuron.2020.09.015. URL https://www.sciencedirect.com/science/article/pii/S0896627320307157.

Jeffrey L. Krichmar. A biologically inspired action selection algorithm based on principles of neuromodulation. In *The 2012 International Joint Conference on Neural Networks (IJCNN)*, pages 1–8, June 2012. doi:10.1109/IJCNN.2012.6252633. URL https://ieeexplore.ieee.org/abstract/document/6252633. ISSN: 2161-4407.

Emre O. Neftci, Hesham Mostafa, and Friedemann Zenke. Surrogate Gradient Learning in Spiking Neural Networks: Bringing the Power of Gradient-Based Optimization to Spiking Neural Networks. *IEEE Signal Processing Magazine*, 36(6):51–63, November 2019. ISSN 1558-0792. doi:10.1109/MSP.2019.2931595. Conference Name: IEEE Signal Processing Magazine.

Friedemann Zenke, Sander M. Bohté, Claudia Clopath, Iulia M. Comşa, Julian Göltz, Wolfgang Maass, Timothée Masquelier, Richard Naud, Emre O. Neftci, Mihai A. Petrovici, Franz Scherr, and Dan F. M. Goodman. Visualizing a joint future of neuroscience and neuromorphic engineering. *Neuron*, 109(4):571–575, February 2021. ISSN 0896-6273. doi:10.1016/j.neuron.2021.01.009. URL https://www.sciencedirect.com/science/article/pii/S089662732100009X.

Nicolas Perez-Nieves, Vincent C. H. Leung, Pier Luigi Dragotti, and Dan F. M. Goodman. Neural heterogeneity promotes robust learning. *Nature Communications*, 12(1):5791, October 2021. ISSN 2041-1723. doi:10.1038/s41467-021-26022-3. URL https://www.nature.com/articles/s41467-021-26022-3. Number: 1 Publisher: Nature Publishing Group.

Catherine D. Schuman, Shruti R. Kulkarni, Maryam Parsa, J. Parker Mitchell, Prasanna Date, and Bill Kay. Opportunities for neuromorphic computing algorithms and applications. *Nature Computational Science*, 2(1):10–19, January 2022. ISSN 2662-8457. doi:10.1038/s43588-021-00184-y. URL https://www.nature.com/articles/s43588-021-00184-y. Publisher: Nature Publishing Group.

Dhireesha Kudithipudi, Catherine Schuman, Craig M. Vineyard, Tej Pandit, Cory Merkel, Rajkumar Kubendran, James B. Aimone, Garrick Orchard, Christian Mayr, Ryad Benosman, Joe Hays, Cliff Young, Chiara Bartolozzi, Amitava Majumdar, Suma George Cardwell, Melika Payvand, Sonia Buckley, Shruti Kulkarni, Hector A. Gonzalez, Gert Cauwenberghs, Chetan Singh Thakur, Anand Subramoney, and Steve Furber. Neuromorphic computing at scale. *Nature*, 637(8047): 801–812, January 2025. ISSN 1476-4687. doi:10.1038/s41586-024-08253-8. URL https://www.nature.com/articles/s41586-024-08253-8. Publisher: Nature Publishing Group.

Friedemann Zenke and Surya Ganguli. SuperSpike: Supervised learning in multilayer spiking neural networks. *Neural Computation*, 2018. ISSN 1530888X. doi:10.1162/neco_a_01086.

Diederik P. Kingma and Jimmy Ba. Adam: A Method for Stochastic Optimization, January 2017. URL http://arxiv.org/abs/1412.6980. arXiv:1412.6980 [cs].

Benjamin Cramer, Yannik Stradmann, Johannes Schemmel, and Friedemann Zenke. The Heidelberg Spiking Data Sets for the Systematic Evaluation of Spiking Neural Networks. *IEEE Transactions on Neural Networks and Learning Systems*, 33(7):2744–2757, July 2022. ISSN 2162-2388. doi:10.1109/TNNLS.2020.3044364. URL https://ieeexplore.ieee.org/document/9311226.

Pete Warden. Speech Commands: A Dataset for Limited-Vocabulary Speech Recognition, April 2018. URL http://arxiv.org/abs/1804.03209. arXiv:1804.03209 [cs].

Arnon Amir, Brian Taba, David Berg, Timothy Melano, Jeffrey McKinstry, Carmelo Di Nolfo, Tapan Nayak, Alexander Andreopoulos, Guillaume Garreau, Marcela Mendoza, Jeff Kusnitz, Michael Debole, Steve Esser, Tobi Delbruck, Myron Flickner, and Dharmendra Modha. A Low Power, Fully Event-Based Gesture Recognition System. In *2017 IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 7388–7397, July 2017. doi:10.1109/CVPR.2017.781. URL https://ieeexplore.ieee.org/document/8100264. ISSN: 1063-6919.

Adam Paszke, Sam Gross, Francisco Massa, Adam Lerer, James Bradbury, Gregory Chanan, Trevor Killeen, Zeming Lin, Natalia Gimelshein, Luca Antiga, Alban Desmaison, Andreas Köpf, Edward Yang, Zach DeVito, Martin Raison, Alykhan Tejani, Sasank Chilamkurthy, Benoit Steiner, Lu Fang, Junjie Bai, and Soumith Chintala. PyTorch: An Imperative Style, High-Performance Deep Learning Library, December 2019. URL http://arxiv.org/abs/1912.01703. arXiv:1912.01703 [cs, stat].

Robert W. Peters, Brian C. J. Moore, and Thomas Baer. Speech reception thresholds in noise with and without spectral and temporal dips for hearing-impaired and normally hearing people. *The*

Journal of the Acoustical Society of America, 103(1):577–587, January 1998. ISSN 0001-4966. doi:10.1121/1.421128. URL https://doi.org/10.1121/1.421128.

Christian Lorenzi, Gaëtan Gilbert, Héloïse Carn, Stéphane Garnier, and Brian C. J. Moore. Speech perception problems of the hearing impaired reflect inability to use temporal fine structure. *Proceedings of the National Academy of Sciences*, 103(49):18866–18869, December 2006. doi:10.1073/pnas.0607364103. URL https://www.pnas.org/doi/abs/10.1073/pnas.0607364103. Publisher: Proceedings of the National Academy of Sciences.

Douglas O'Shaughnessy. Trends and developments in automatic speech recognition research. *Computer Speech & Language*, 83:101538, January 2024. ISSN 0885-2308. doi:10.1016/j.csl.2023.101538. URL https://www.sciencedirect.com/science/article/pii/S0885230823000578.

Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. Attention Is All You Need. *arXiv:1706.03762 [cs]*, December 2017. URL http://arxiv.org/abs/1706.03762. arXiv: 1706.03762.

Michael C. Avery and Jeffrey L. Krichmar. Neuromodulatory Systems and Their Interactions: A Review of Models, Theories, and Experiments. *Frontiers in Neural Circuits*, 11, December 2017. ISSN 1662-5110. doi:10.3389/fncir.2017.00108. URL https://www.frontiersin.org/journals/neural-circuits/articles/10.3389/fncir.2017.00108/full. Publisher: Frontiers.